

Class 2: Setting Up Python

Python environment, Jupyter, VS Code, and virtual environments

- ▶ Install Python and check that it works from the terminal.
- ▶ Install Visual Studio Code and the Python/Jupyter extensions.
- ▶ Create an isolated project environment with `venv`.
- ▶ Install Jupyter and common Python libraries.
- ▶ Understand the alternative: Miniconda and Conda environments.
- ▶ Run the first script and the first notebook.

By the end of this class, you should be able to

- ▶ Explain why local Python projects should use isolated environments.
- ▶ Install and verify Python, pip, VS Code, and Jupyter.
- ▶ Create, activate, use, and deactivate a `venv` environment.
- ▶ Choose between `venv` and Miniconda for a basic Python workflow.

From notebooks to real projects

In this course, Python is not only used to write small examples: it becomes the environment where data, models, APIs, and dashboards live.

- ▶ A working setup is therefore part of the programming workflow, not an administrative detail.
- ▶ If the environment is reproducible, code can be tested, shared, deployed, and debugged much more easily.

Main idea

We create one clean project folder with its own Python interpreter, its own packages, and its own notebook kernel.

- ▶ **Python** runs the code.
- ▶ **VS Code** is the editor where we write scripts and notebooks.
- ▶ `my_first_env` is the isolated environment for this project.
- ▶ **Packages** such as NumPy and pandas are installed inside that environment.
- ▶ **Jupyter** lets us combine code, text, plots, and results interactively.

Step 1 — Install Python

1. Go to the official Python website and download a recent Python 3 version.
2. On Windows, check **Add python.exe to PATH** before clicking **Install Now**.
3. On macOS/Linux, Python may already exist, but installing a recent Python 3 version is still recommended.
4. Open a terminal and verify the installation.

```
python --version  
# or, on some systems:  
python3 --version
```

Step 2 — Check pip

- ▶ pip is the standard tool used to install Python packages.
- ▶ Always check pip from the same Python installation you plan to use.

```
python -m pip --version
```

Why use `python -m pip`?

It reduces confusion when multiple Python versions are installed: the selected Python runs its own package installer.

Step 3 — Install Visual Studio Code

1. Download and install Visual Studio Code.
2. Open VS Code.
3. Go to the Extensions panel.
4. Install the Microsoft **Python** extension.
5. Install the Microsoft **Jupyter** extension.

Course relevance

VS Code lets us edit scripts, open terminals, run notebooks, and select the correct Python environment from one place.

Step 4 — Create a project folder

- ▶ Keep each project in its own folder.
- ▶ Open a terminal and move into that folder before creating the environment.

```
mkdir ml_course  
cd ml_course
```

Mental model

Everything we install next belongs to this project, not to the whole computer.

Step 5 — Create a virtual environment with venv

- ▶ venv is included with Python.
- ▶ We will call our environment `my_first_env`, so its purpose is visible in the project folder.

```
python -m venv my_first_env  
# or, on some macOS/Linux systems:  
python3 -m venv my_first_env
```

Result

Your project now contains an isolated Python environment inside `my_first_env`.

Step 6 — Activate the environment

Windows

```
my_first_env\Scripts\activate
```

PowerShell alternative

```
my_first_env\Scripts\Activate.ps1
```

macOS/Linux

```
source my_first_env/bin/activate
```

You should see

```
(my_first_env) ...
```

Step 7 — Install packages

- ▶ With the environment active, install the libraries needed for the course.
- ▶ These packages are installed inside `my_first_env`, not globally.

```
python -m pip install numpy pandas matplotlib scikit-learn
```

```
python -m pip list
```

Why not just `pip install ...`?

`pip install ...` may use a different Python if several versions or environments exist. `python -m pip ...` says explicitly: install into the Python environment I am currently running.

Step 8 — Install Jupyter

- ▶ Jupyter lets us write code, text, formulas, and plots in the same document.
- ▶ JupyterLab is the modern notebook interface.

```
python -m pip install jupyterlab  
jupyter lab
```

First check

Create a notebook, select the project kernel, and run `print("Hello world!")`.

Step 9 — Test Python in a notebook

```
import numpy as np

x = np.array([1, 2, 3, 4])
print(x)
print(x.mean())
```

Expected output

If the cell runs and prints 2.5, Python, NumPy, Jupyter, and the environment are working together.

Step 10 — Select the environment in VS Code

1. Open the project folder in VS Code.
2. Open the command palette: `Ctrl/Cmd + Shift + P`.
3. Search for Python: `Select Interpreter`.
4. Choose the interpreter inside `my_first_env` (`my_first_env/bin/python`).
5. For notebooks, click **Select Kernel** and choose the same environment.

Important

The terminal, Python scripts, and notebooks should all use the same environment.

Step 11 — Run the first Python script

- ▶ Create a file called `hello.py`.
- ▶ Write a few lines of Python.
- ▶ Run it from the VS Code terminal.

```
name = "Alice"  
print("Hello", name)
```

```
python hello.py
```

Step 12 — Deactivate and reactivate

- ▶ When finished, deactivate the environment.
- ▶ The environment is not deleted: it is only switched off.

```
deactivate
```

Next time

Return to the project folder and activate `my_first_env` again before working.

- ▶ Miniconda is a small Python distribution with the `conda` environment manager.
- ▶ It is very common in data science and scientific computing.
- ▶ It can manage Python versions and complex binary dependencies.
- ▶ It is slightly heavier conceptually than `venv`, but useful for scientific Python stacks.

Practical rule

Use `venv` for simple Python projects; consider Conda when scientific dependencies become difficult to install.

Creating a Conda environment

Download and install:

<https://www.anaconda.com/docs/getting-started/installation>

```
conda create -n mlcourse python=3.13
conda activate mlcourse
conda install numpy pandas matplotlib scikit-learn jupyter
jupyter lab
```

```
conda env remove -n mlcourse
```

Same idea, different tool

Both venv and Conda create isolated environments. The goal is the same: make the project reproducible and avoid dependency conflicts.

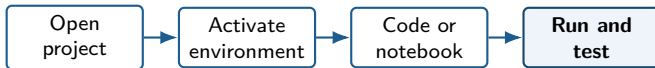
venv

- ▶ Included with Python.
- ▶ Uses pip.
- ▶ Lightweight and simple.
- ▶ Good first option for teaching.

Miniconda

- ▶ Separate installation.
- ▶ Uses conda and sometimes pip.
- ▶ More features for scientific stacks.
- ▶ Useful for complex scientific dependencies.

- ▶ **python not found**: check PATH, try `python3`, or reinstall Python.
- ▶ **Package not found**: activate the environment and install the package there.
- ▶ **Notebook uses wrong packages**: select the correct Jupyter kernel.
- ▶ **Activation blocked on Windows**: use Command Prompt or adjust PowerShell execution policy for local scripts.



The habit to build

Never install packages randomly. First activate the project environment, then install and run code there.

Sharing your environment

venv + pip

```
# Export dependencies
python -m pip freeze > requirements.txt

# Friend recreates the environment
python -m venv myenv
source myenv/bin/activate
python -m pip install -r requirements.txt
```

Share

Project files + requirements.txt +
Python version.

Important

Do **not** share the environment folder itself. Share the environment specification and recreate it on the other machine.

Conda

```
# Export environment
conda env export --from-history >
    environment.yml

# Friend recreates it
conda env create -f environment.yml
conda activate <environment_name>
```

Share

Project files + environment.yml.

Questions you should be able to answer

- ▶ Why should every Python project have its own environment?
- ▶ What is the difference between Python, pip, VS Code, and Jupyter?
- ▶ How do you create and activate a `venv` environment?
- ▶ How do you install a package in the active environment?
- ▶ When might Miniconda be useful instead of `venv`?

Question

A model works on your laptop, but not on a colleague's machine. What could have gone wrong?

- ▶ Different Python versions.
- ▶ Missing or incompatible packages.
- ▶ Wrong VS Code interpreter or notebook kernel.
- ▶ No documented environment setup.

Useful references

Python downloads; Visual Studio Code; Python venv documentation; Jupyter installation guide; Conda documentation.

- ▶ A reliable Python workflow starts with a reproducible Python environment.
- ▶ `venv` is the simplest way to isolate packages for one project.
- ▶ VS Code and Jupyter must point to the same environment to avoid confusion.
- ▶ Miniconda is a useful alternative, especially for scientific Python dependencies.

Before next class

Install Python, create one project folder, activate an environment, and run one notebook successfully.

- ▶ Once this works, we can move from setup to real data handling.
- ▶ The next step is using `pandas` to load, inspect, clean, and visualize data.